beta_via_approx_dp

Aishwarya Ramasethu (@aramasethu), Yu-Ju Ku (@yuju1998), Jordan Awan, Michael Shoemate (@Shoeboxam)

November 5, 2026

This proof resides in **"contrib"** because it has not completed the vetting process.

1 Hoare Triple

Precondition

Compiler-verified

- Arguments `epsilon`, `delta`, and `alpha` of type `f64`

Caller-verified

The source parameters satisfy finite nonnegative ϵ, $\delta \in [0, 1]$, and $\alpha \in [0, 1]$. The function must complete successfully.

Pseudocode

```
def beta_via_approx_dp(epsilon: f64, delta: f64, alpha: f64) -> Fallible[f64]:
    check_epsilon(epsilon)
    check_delta(delta)
    check_alpha(alpha)
    if alpha == 1.0:
        return 0.0
    one_minus_delta = RBig(1) - RBig(delta)
    if alpha == 0.0:
        return f64.neg_inf_cast(one_minus_delta)
    epsilon = SInterval.point(epsilon)
    exp_eps_up = RBig(epsilon.exp().upper_f64())
    exp_neg_eps_down = RBig((-epsilon).exp().lower_f64())
    alpha = RBig(alpha)
    t1 = one_minus_delta - exp_eps_up * alpha
    base = max(one_minus_delta - alpha, RBig(0))
    t2 = exp_neg_eps_down * base
    return clamp(f64.neg_inf_cast(max(t1, t2, RBig(0))), 0.0, 1.0)
```

Postcondition

Theorem 1.1. For all $\alpha \in [0, 1]$, `beta_via_approx_dp` returns a value no greater than the exact approximate-DP tradeoff curve $f_{\epsilon,\delta}(\alpha)$.

Definition 1.2. For a fixed approximate-DP certificate with parameters ϵ and δ, define

$$f_{\epsilon,\delta}(\alpha) = \max(0, 1 - \delta - \exp(\epsilon)\alpha, \exp(-\epsilon)\max(1 - \delta - \alpha, 0)). \tag{1}$$

Proof. The function directly computes exact rational conversions of directed interval endpoints satisfying

$$\texttt{exp_eps_up} \geq \exp(\epsilon), \qquad \texttt{exp_neg_eps_down} \leq \exp(-\epsilon).$$

The pseudocode uses these coefficients with exact rational arithmetic.

For $\alpha \in [0, 1]$, the first affine branch is no greater than $1 - \delta - \exp(\epsilon)\alpha$ because `exp_eps_up` is an upper bound and α is nonnegative. The second branch is similarly no greater than its exact counterpart because `exp_neg_eps_down` is a lower bound. Taking the maximum with zero preserves the pointwise lower bound. The final conversion to `f64` rounds downward. The endpoints return zero at $\alpha = 1$ and a downward-rounded $1 - \delta$ at $\alpha = 0$, which satisfy the same bound. □